Project - High Level Design

On

HealthCare Content Generator

Course Name: GenAI

Institution Name: Medi-caps University – DataGami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Kashish Bilala	EN24CA5030079
2.	Komolika Vyas	EN24CA5030083
3.	Aayush	EN22IT301004
4.	Danish Shaikh	EN22IT301032
5.	Prakhar Jain	EN24CA5030122

Group Name: Group – G-04D12

Project Number:GAI-04

Industry Mentor Name:

University Mentor Name: Ajaj Khan

Academic Year:2025-2026

Table of Contents

Page No.

1. Introduction. 3 - 4
   1.1. Scope of the document. 3
   1.2. Intended Audience 4
   1.3. System overview. 4

2. System Design. 5 - 12
   2.1. Application Design 5
   2.2. Process Flow. 6-7
   2.3. Information Flow. 8
   2.4. Components Design 9
   2.5. Key Design Considerations 10
   2.6. API Catalogue. 11

3. Data Design. 13 - 15
   3.1. Data Model 13
   3.2. Data Access Mechanism 14
   3.3. Data Retention Policies 14
   3.4. Data Migration 15

4. Interfaces 16

5. State and Session Management 17

6. Caching 18

7. Non-Functional Requirements 19 - 20
   7.1. Security Aspects 19
   7.2. Performance Aspects 20

8. References 21

1.Introduction

The Healthcare Planning Assistant Agent is an AI-based system designed to assist users in planning healthcare tasks efficiently. It uses LangChain along with the Groq API to break down user queries into smaller steps and generate structured plans.

The system consists of a frontend for user interaction, a FastAPI backend for request handling, and a planner agent that performs multi-step reasoning. It interacts with tools such as doctor availability, treatment options, and cost estimation to provide organized and intelligent outputs.

This project demonstrates how Agentic AI can automate decision-making and simplify healthcare workflows.

1.1 Scope of the Project

This document provides a detailed low-level design (LLD) for the Healthcare Planning Assistant Agent. It outlines system architecture, component-level design, workflows, APIs, and implementation details required to build an intelligent agent capable of planning healthcare-related tasks through multi-step reasoning. The document focuses on:

- Task decomposition logic
- Agent orchestration using frameworks like LangChain
- Integration with LLM APIs
- Mock healthcare tools and scheduling mechanisms

1.2 Intended Audience

The Healthcare Planning Assistant Agent is designed for a wide range of users who require assistance in planning healthcare-related tasks. The primary audience includes patients and general users who want quick and structured information about treatment options, doctor availability, and estimated costs. It can also be useful for students and learners who are exploring applications of AI in the healthcare domain.

Additionally, the system can benefit healthcare professionals as a supportive tool for basic planning and decision-making. Developers and researchers interested in Agentic AI and intelligent systems can also use this project as a reference for building similar applications. Overall, the system is intended for anyone seeking an easy-to-use, AI-powered solution for healthcare planning.

1.3 System Overview

The Healthcare Planning Assistant Agent is an intelligent system that:

- Accepts a high-level healthcare goal (e.g., "Find Treatment Options for Diabetes")
    - Uses an LLM-powered reasoning loop to:
    - Break down goals into sub-tasks
    - Validate required resources
    - Manage dependencies
    - Generate optimized execution plans

The system leverages:

- Large Language Models (LLMs)
- Agent frameworks (LangChain)
- Mock healthcare APIs (e.g., doctor availability, treatment database)

2. System Design

The **Healthcare Planning Assistant Agent** follows a modular system design where the user interacts with a frontend interface built using HTML, CSS, and JavaScript to submit healthcare-related queries. These requests are processed by a FastAPI backend, which validates the input and forwards it to the planner agent. The planner agent, developed using LangChain, performs multi-step reasoning by breaking the query into smaller tasks and deciding the sequence of execution. It interacts with various tools such as doctor availability, treatment options, and cost estimation to gather relevant information. For intelligent decision-making and response generation, the system integrates the Groq API. The final structured healthcare plan is then sent back through the backend to the frontend, where it is displayed to the user. This design ensures efficient task handling, scalability, and dynamic planning.

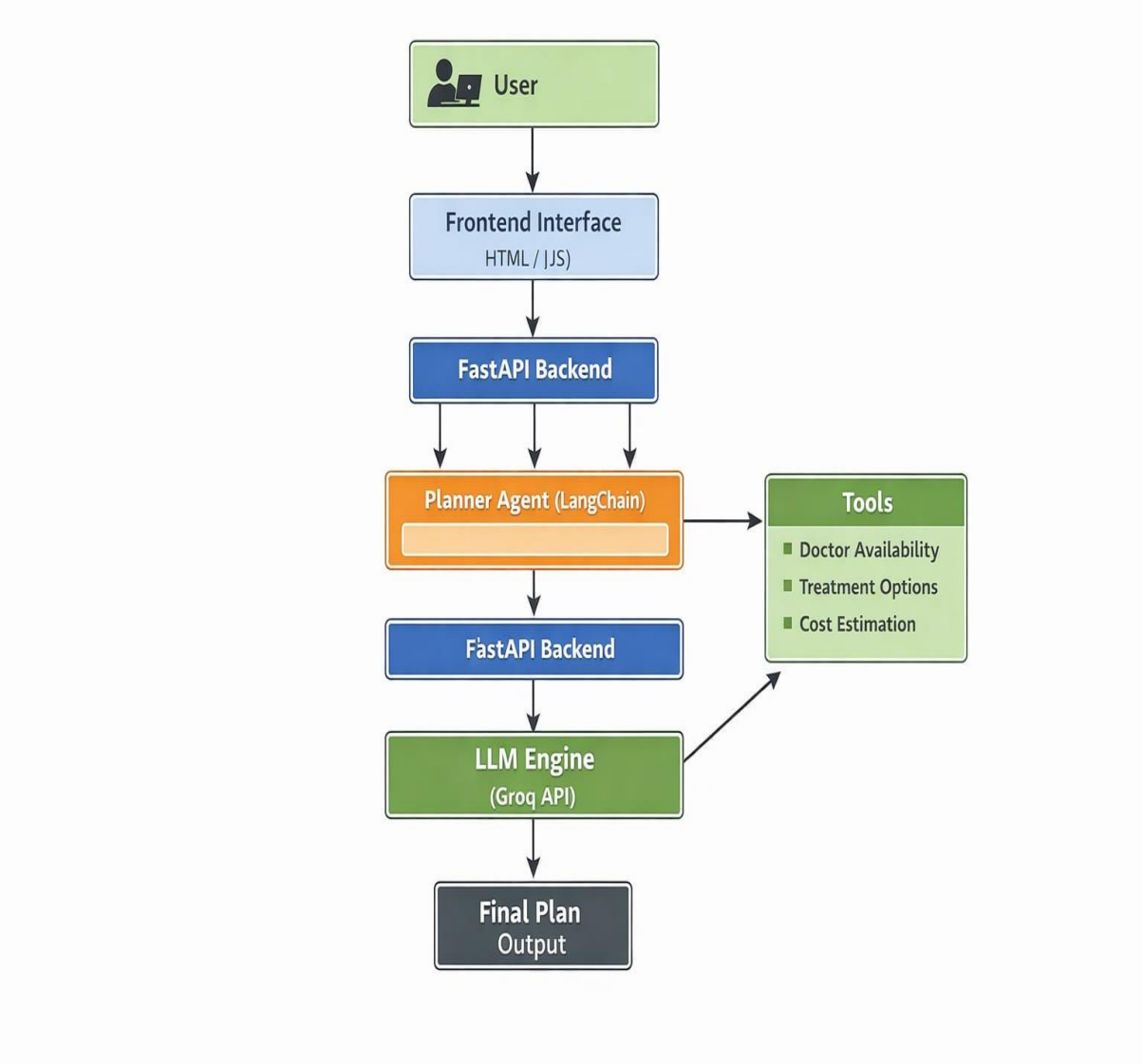
2.1 Application Design

The system follows a modular agent-based architecture where a central Planner Agent handles reasoning and task orchestration.

- The Planner Agent interprets user goals and decomposes them into smaller tasks
- Frameworks like LangChain or CrewAI are used to manage agent workflows
- The system integrates with LLM services via Groq API
- External tools (mock healthcare APIs) are used to validate and execute tasks

Key Features:

- Modular and scalable design
- Supports dynamic planning and execution

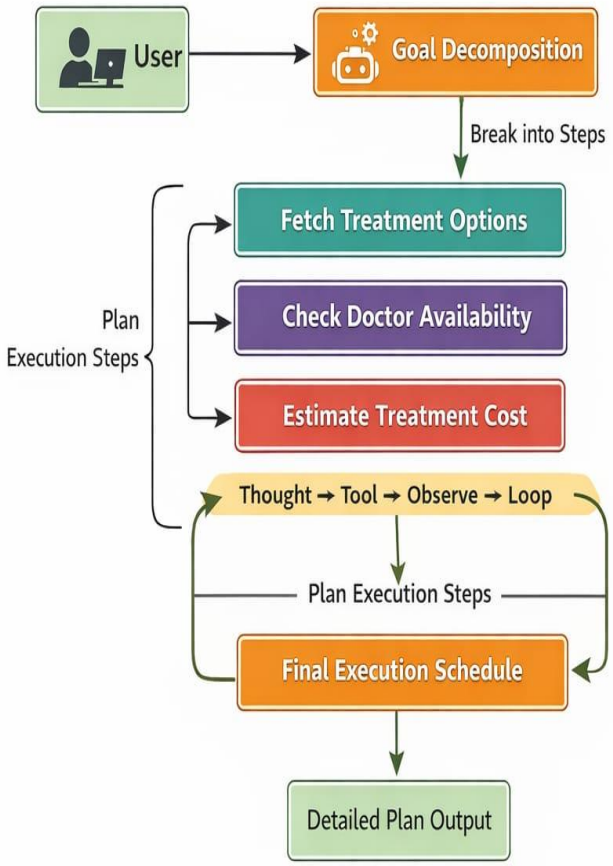


2.2 Process Flow

The system follows a step-by-step execution pipeline:

- User submits a high-level healthcare goal
- Planner Agent analyzes the input
- Goal is decomposed into smaller tasks
- Required resources are identified
- Tools/APIs are called for validation

- Task dependencies are resolved
- Execution plan is generated
- Scheduler assigns order and timeline
- Final plan is returned to the user



Healthcare Planning Assistant Agent Process Flow

2.3 Information Flow

Input:

- User query (goal)
- Optional patient or context data

Internal Flow:

- Input → LLM Processing → Task Decomposition → Tool Interaction → Plan Generation

Output:

- Structured execution plan including:
  - Task list
  - Dependencies
  - Execution schedule

2.4 Component Design

The system consists of the following core components:

1. Planner Agent

- Main intelligence unit
- Performs reasoning and planning

2. Task Manager

- Tracks tasks, status, and dependencies

3. Tool Interface Layer

- Connects to mock healthcare services

- Simulates real-world APIs

4. Execution Engine

- Executes tasks in correct order
- Handles failures and retries

5. LLM Wrapper

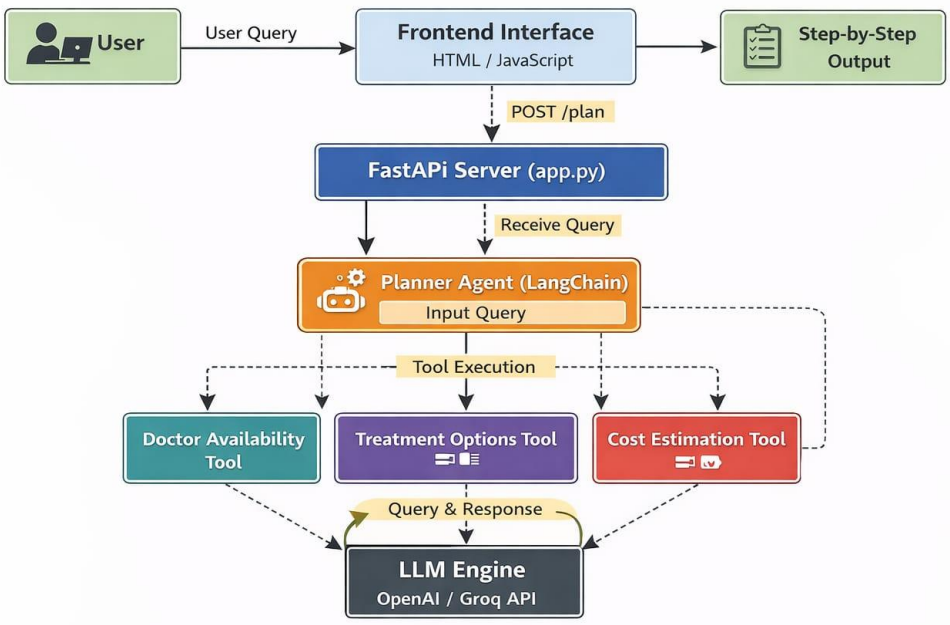
- Manages communication with LLM APIs
- Handles prompts and responses

6. Memory Module

- Stores context, history, and intermediate results

7. Scheduler

- Optimizes task sequence and timing



Healthcare Planning Assistant Agent Component Flow

2.5 Key Design Considerations

1. Scalability

- Modular architecture supports expansion
- Can be extended to microservices

2. Reliability

- Error handling and retry mechanisms
- Validation before execution

3. Explainability

- Transparent reasoning steps
- Traceable decision-making

4. Security

- Safe handling of healthcare data
- API authentication and access control

5. Extensibility

- Easy addition of new tools/services

6. Performance

- Parallel execution where possible
- Caching frequently used results

2.6 API Catalogue

1. Submit Goal API

POST /submit-goal

• Accepts user goal

• Returns plan ID

2. Get Plan API

GET /plan/{plan_id}

• Fetches treatment options

3. Treatment API (Mock Tool)

GET /treatments?condition={condition}

• Returns treatment details based on condition

4. Doctor Availability API

GET /doctors?specialization={type}

• Returns available doctors

5. Task Status API

GET /task-status/{task_id}

• Provides task progress

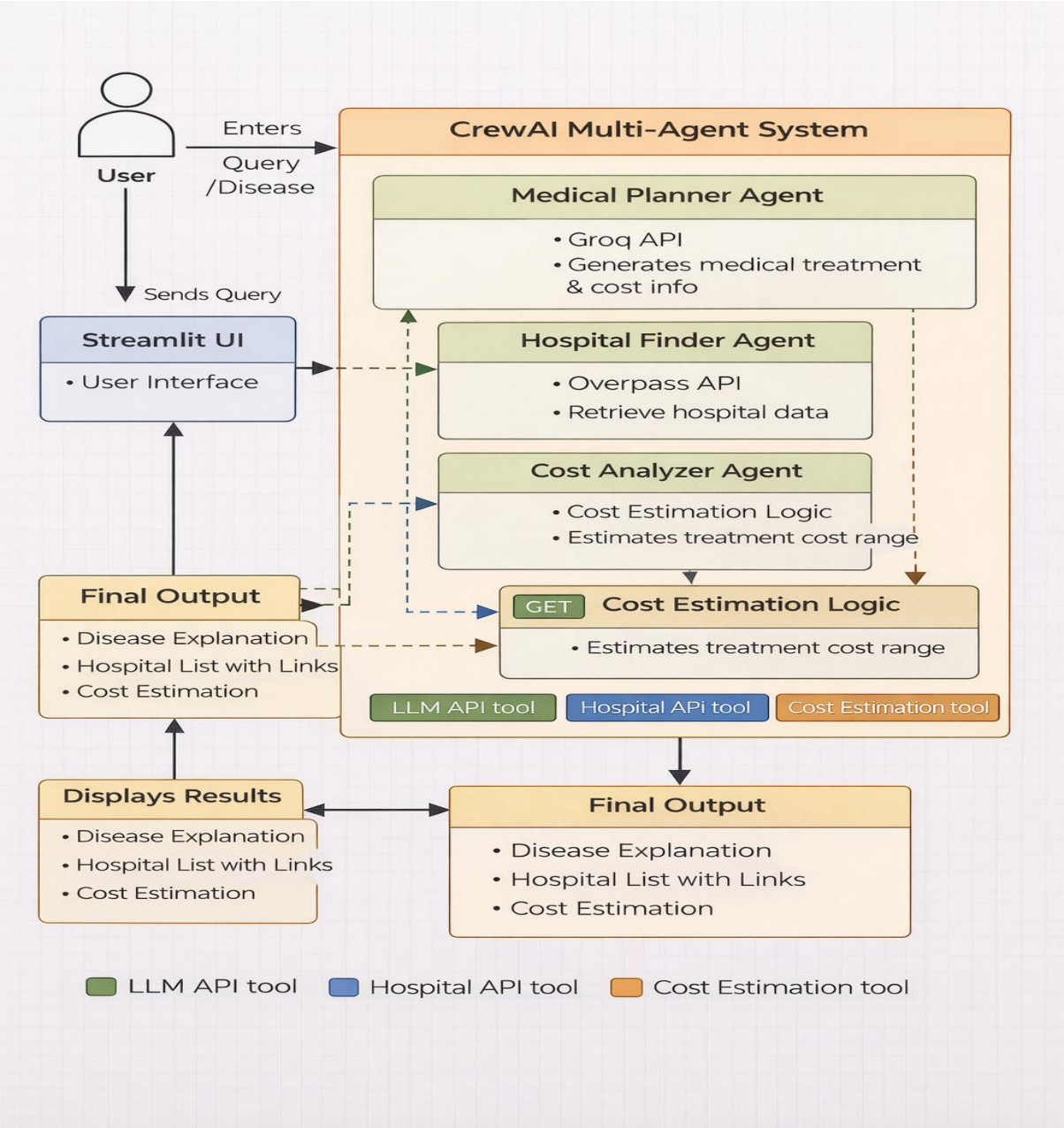
6. Execute Plan API

POST /execute-plan

• Triggers execution of generated plan

• Returns generated execution plan

2.7 UML Diagram



3.Data Design

The data design of the Healthcare Planning Assistant Agent is based on a structured flow of information using JSON format. The user provides input through a frontend built with HTML, CSS, and JavaScript, which sends the query to the backend developed in Python using FastAPI. The backend processes this data and forwards it to the planner agent, where the query is decomposed into smaller tasks. The agent interacts with various tools to retrieve data such as treatment options, doctor availability, and cost estimation. These intermediate results are processed with the help of the Groq API to generate a meaningful response. The final output is structured into a clear healthcare plan and sent back to the frontend for display. Currently, the system uses temporary runtime data and does not include a persistent database.

3.1 Data Model

The data model of the Healthcare Planning Assistant Agent is designed to handle structured information in a simple and efficient manner using JSON-based objects. The primary data entity is the user query, which contains the healthcare-related request entered through the frontend. This input is mapped to a request model in the Python backend, where it is processed and passed to the planner agent. The agent internally generates a task model that represents the breakdown of the query into smaller actionable steps.

Each task interacts with specific tool models that return structured responses such as treatment details, doctor availability, and cost estimates. These intermediate data elements are then combined into a response model, which represents the final healthcare plan. The system follows a flow of input model → task model → tool response model → final output model. Since the project does not use a database, all data is handled dynamically during runtime without persistent storage.

3.2 Data Access Mechanism

The Healthcare Planning Assistant Agent uses a request–response based data access mechanism. The user input is sent from the frontend (HTML, CSS, JavaScript) to the Python FastAPI backend through REST APIs. The backend processes the request and passes it to the planner agent, which accesses required data through integrated tools such as treatment information, doctor availability, and cost estimation. These tools act as data sources and return structured responses to the agent. The agent also interacts with the Groq API for intelligent processing. Since the system does not use a database, all data is accessed dynamically during runtime and returned to the frontend as a JSON response.

3.3 Data Retention Policies

The Healthcare Planning Assistant Agent follows a minimal data retention policy, as it does not store user data permanently. All inputs provided by the user are processed in real-time by the Python FastAPI backend and are used only for generating the healthcare plan. The system does not maintain any database or long-term storage, ensuring that user queries and responses exist only during the active session. After the response is generated and sent back to the frontend, the data is automatically discarded from memory.
This approach helps maintain user privacy and reduces the risk of data misuse or unauthorized access. Additionally, since no sensitive healthcare information is stored, the system remains lightweight and secure. The design ensures that data handling is temporary, efficient, and aligned with basic privacy principles.

3.4 Data Migration

The Healthcare Planning Assistant Agent does not currently require a data migration process, as it does not use any database or persistent storage. All data is processed dynamically at runtime and is not stored for future use. Therefore, there is no need for transferring or migrating data between systems or storage layers.

However, if the system is upgraded in the future to include a database or knowledge base, data migration may become necessary. In such cases, healthcare data such as patient records, treatment information, or historical queries can be migrated into a structured database or vector storage system. This would enable better data management, scalability, and support for advanced features like Retrieval-Augmented Generation (RAG).

4. Interfaces

The Healthcare Planning Assistant Agent is designed with multiple interfaces that enable smooth communication between different components of the system. The primary interface is the user interface, developed using HTML, CSS, and JavaScript. This interface allows users to enter healthcare-related queries and view the generated treatment plans in a simple and interactive manner. It acts as the entry point of the system and ensures user-friendly interaction.

The frontend communicates with the backend through RESTful API calls. The backend interface is implemented using Python with FastAPI, which handles incoming HTTP requests, validates input data, and returns structured JSON responses. This layer acts as a bridge between the user interface and the core logic of the system, ensuring efficient request processing and response delivery.

The planner agent interface, built using LangChain, is responsible for managing the internal workflow of the system. It receives processed input from the backend, breaks it into smaller tasks, and coordinates with different tools to gather necessary information. This interface enables multi-step reasoning and dynamic decision-making within the system.

Additionally, the system integrates with external services through the Groq API, which provides advanced natural language processing capabilities. This interface helps the agent understand user queries and generate intelligent, context-aware responses.

The tools interface is another important component, which includes modules for doctor availability, treatment options, and cost estimation. These tools act as internal data providers and are accessed by the planner agent during execution. All these interfaces work together to ensure seamless data flow, modularity, scalability, and efficient system performance.

5. State and Session Management

The Healthcare Planning Assistant Agent follows a stateless architecture for handling user requests. Each request from the frontend is processed independently by the Python FastAPI backend without maintaining any previous interaction history. This means that the system does not store session data or user context across multiple requests, ensuring simplicity and scalability.

During a single request, the state is maintained temporarily within the planner agent, where the query is processed through multiple steps such as task decomposition, tool interaction, and response generation. Once the final output is generated and returned to the frontend, this temporary state is cleared from memory.

Since the system does not implement session tracking or persistent storage, it does not support features like conversation history or user-specific personalization. However, this stateless approach improves performance, reduces memory usage, and enhances data privacy. In future enhancements, session management techniques such as token-based sessions or context storage can be added to support multi-turn interactions and personalized healthcare planning.

6. Caching

The system implements a limited and controlled caching mechanism to improve performance and reduce response time. Since the application primarily processes data in real-time using Generative AI, caching is not extensively used for storing long-term data. However, temporary caching may be applied to store frequently accessed or recently generated responses within a short duration. This helps in reducing repeated processing of similar requests and enhances system efficiency.

Caching is mainly utilized at the application level, where intermediate results such as generated summaries or processed prompts may be temporarily stored during a user session. This allows faster retrieval of results when the same data is required again within the session, such as during report generation or page refresh.

The system ensures that cached data is short-lived and automatically cleared after a defined time period or once the session ends. This approach prevents the storage of outdated or sensitive healthcare information and maintains data privacy. Additionally, caching mechanisms are designed to avoid inconsistencies by ensuring that only valid and relevant data is temporarily stored.

Overall, the caching strategy is optimized to balance performance improvement and data security, ensuring faster response times while maintaining the integrity and confidentiality of user data.

7. Non-Functional Requirements

The Healthcare Planning Assistant Agent is designed to ensure efficient performance and reliability. The system should provide fast response times by processing user queries in real time through the FastAPI backend. It must be scalable to handle multiple users without performance issues.
The system ensures reliability by generating consistent outputs using the planner agent and the Groq API. It also maintains security and privacy by not storing user data permanently. Additionally, the system is user-friendly and modular, allowing easy maintenance and future enhancements.

7.1 Security Aspects

The Healthcare Planning Assistant Agent is designed with basic security measures to protect user data and ensure safe system operation. The system does not store user information permanently, which reduces the risk of data breaches and unauthorized access. All data is processed temporarily during runtime and discarded after generating the response.
Communication between the frontend and the Python FastAPI backend is handled through secure API requests, ensuring controlled data exchange. Sensitive information such as API keys for the Groq API is stored securely using environment variables and is not exposed in the source code.
Input validation is performed at the backend to prevent invalid or malicious data from affecting the system. Additionally, the modular architecture allows future integration of advanced security features such as authentication, authorization, and encrypted communication. Overall, the system follows basic security practices to maintain data privacy and system integrity.

7.2 Performance Aspects

The Healthcare Planning Assistant Agent is designed to deliver efficient and responsive performance. The system processes user queries in real time using a Python FastAPI backend, ensuring quick request handling and low latency. The use of a planner agent allows structured task execution, which improves the efficiency of generating healthcare plans.
The integration with the Groq API enables fast inference and reduces

response time for complex queries. Since the system does not use a database, it avoids delays related to data storage and retrieval. The stateless architecture further enhances performance by handling each request independently.

Overall, the system is optimized for speed, scalability, and efficient resource utilization, ensuring smooth performance even with multiple user requests.

References

1. LangChain Documentation – https://docs.langchain.com/
2. FastAPI Documentation – https://fastapi.tiangolo.com/
3. Groq API Documentation – https://console.groq.com/docs
4. Python Official Documentation – https://docs.python.org/3/
5. JavaScript MDN Web Docs – https://developer.mozilla.org/
6. HTML & CSS Documentation – https://developer.mozilla.org/